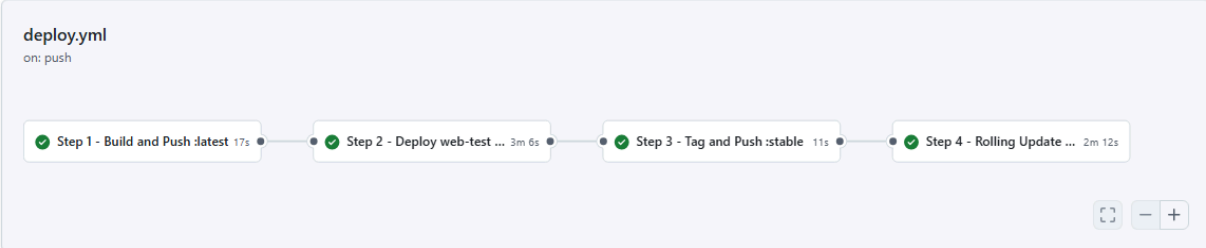
GitHub Actions Workflow 개선

업무 영역	Devops 자동화
진행현황	대기
담당자	동욱 김
진행률	0
설명	1. GitHub Actions → EC2 SSH 배포 개선
주차	4주차

0. 개요



- 3주차 공통 실습에 작성한 `deploy.yml` 파일을 깃허브 액션으로 실행
- step 1: 17초
  step 2: **3분 8초**
  step 3: 11초
  step 4: **2분 12초**
- step2, step4의 execute 부분에서 2분 30초 ~ 4분의 시간 소요. 이를 개선하여 2분까지 단축 시도

▼ 1. step2 개선

```
#!/bin/bash
set -e
echo "Deploy web-test and run health check"

# 1. web-test 인스턴스 생성
```

```
instance_id=$(aws ec2 run-instances \
    --launch-template LaunchTemplateName="lt-k" \
    --query 'Instances[0].InstanceId' \
    --output text)
echo "Web-test instance id: $instance_id"

# 2. 상태 ok 대기
aws ec2 wait instance-status-ok --instance-ids "$instance_id"
echo "Ec2 system status: ok"

# 3. web-test ip 조회
private_ip=$(aws ec2 describe-instances \
    --instance-ids "$instance_id" \
    --query 'Reservations[0].Instances[0].PrivateIpAddress' \
    --output text)
echo "Web-test private ip: $private_ip"

# 4. 헬스 체크 루프 (15초 * 20회)
for i in $(seq 20); do
        echo "Health check attempt: $i/20"
        health_status=$(ssh -o StrictHostKeyChecking=no -i "~/.ssh/web-
key.pem" ubuntu@"$private_ip" \
        "curl -s -o /dev/null -w '%{http_code}' http://localhost/health" || ec
ho "000")
        echo "App health status: $health_status"

        # 헬스 체크 통과
        if [ "$health_status" = "200" ]; then
        echo "Health check passed ✅"
        aws ec2 terminate-instances --instance-ids "$instance_id"
        rm -f ~/.ssh/web-key.pem
        echo "Instance terminated and key removed"
        break
    fi
        # 헬스 체크 실패
        if [ "$i" -eq 20 ]; then
        echo "Health check failed ❌"
        aws ec2 terminate-instances --instance-ids "$instance_id"
```

```
        rm -f ~/.ssh/web-key.pem
        echo "Instance terminated and key removed"
        exit 1
    fi
    sleep 10
done
```

기존 코드의 문제점

1. `wait instance-status-ok` 커맨드가 15초마다 상태를 확인하여 시간 지연
2. 헬스 체크 루프가 10초 * 20회 = 최악 200초로 시간 지연

코드 개선점

1. ok 상태를 기다리지 않고 `instance-running` 상태면 바로 SSH 연결 시도
2. 헬스 체크 루프 단위를 3초 * 30회 = 90초로 단축
3. 사용자 종료, JOB 최소 등의 경우에도 코드 종료 및 정리를 보장하기 위해 `set -e` 대신 `trap` 과 `set -Eeuo pipefail` 사용

개선 코드

```
set -Eeuo pipefail # 사용자 종료, JOB 최소 등의 경우에도 코드 종료 및 정리를 보장

echo "Deploy web-test and run health check (fast mode)"

terminate() {
  local id="${1:-}"
  if [[ -n "${id}" ]]; then
    echo "Terminating instance ${id}..."
    aws ec2 terminate-instances --instance-ids "${id}" >/dev/null
  fi
}
cleanup() {
  rm -f ~/.ssh/web-key.pem || true
}
trap 'cleanup' EXIT
```

```
# 1) web-test 인스턴스 생성
instance_id=$(aws ec2 run-instances \
  --launch-template LaunchTemplateName="lt-k" \
  --query 'Instances[0].InstanceId' --output text)
echo "Web-test instance id: ${instance_id}"

# 문제가 생겨도 반드시 종료
trap 'terminate "${instance_id}"; cleanup' ERR INT

# 2) 상태 ok 대기 -> running 상태까지만 대기 (빠름)
aws ec2 wait instance-running --instance-ids "${instance_id}"
echo "Instance state: running"

# 3)  web-test ip 조회
private_ip=$(aws ec2 describe-instances --instance-ids "${instance_i
d}" \
  --query 'Reservations[0].Instances[0].PrivateIpAddress' --output text)
echo "Web-test private ip: ${private_ip}"

# 4) 헬스 체크 루프 ( 3초 * 30회)
echo "Waiting for SSH to be ready..."
deadline=$((SECONDS + 90))
while (( SECONDS < deadline )); do
  if ssh -o StrictHostKeyChecking=no -o ConnectTimeout=2 -i "~/.ssh/w
eb-key.pem" \
       -q ubuntu@"${private_ip}" "exit 0"; then
    echo "SSH ready."
    break
  fi
  sleep 3
done
if (( SECONDS >= deadline )); then
  echo "SSH not ready in time ❌"
  terminate "${instance_id}"
  exit 1
fi

# 5) /health 초단위 헬스 체크(3초 간격, 최대 90초)
```

```
echo "Probing app health..."
deadline=$((SECONDS + 90))
while (( SECONDS < deadline )); do
  code=$(ssh -o StrictHostKeyChecking=no -o ConnectTimeout=2 -i
"~/.ssh/web-key.pem" \
    ubuntu@"${private_ip}" \
    "curl -s -o /dev/null -w '%{http_code}' --connect-timeout 1 --max-tim
e 2 http://localhost/health" \
    || echo "000")
  echo "Health: ${code}"
  if [[ "${code}" == "200" ]]; then
    echo "Health check passed ✅"
    terminate "${instance_id}"
    rm -f ~/.ssh/web-key.pem
    exit 0
  fi
  sleep 3
done

echo "Health check failed ❌"
terminate "${instance_id}"
rm -f ~/.ssh/web-key.pem
exit 1
```

▼ 2. step4 개선

```
#!/bin/bash
set -e
echo "Rolling update ASG"
ASG_NAME="asg-k"
ORIGIN_MIN=0
ORIGIN_MAX=0
ORIGIN_DESIRED=0

# 인스턴스 상태 출력 함수
print_instance_states() {
  echo "현재 인스턴스 상태:"
```

```
  aws autoscaling describe-auto-scaling-groups \
    --auto-scaling-group-names "$ASG_NAME" \
    --query 'AutoScalingGroups[0].Instances[*].[InstanceId, LifecycleStat
e, HealthStatus]' \
    --output text
}

# 기존 용량 복구 함수
restore_capacity() {
  aws autoscaling update-auto-scaling-group \
    --auto-scaling-group-name "$ASG_NAME" \
    --min-size "$ORIGIN_MIN" \
    --max-size "$ORIGIN_MAX" \
    --desired-capacity "$ORIGIN_DESIRED"
  echo "용량 복원 완료: $ORIGIN_MIN(Min), $ORIGIN_MAX(Max), $ORIGIN
_DESIRED(Desired)"
}

# 1. 기존 용량 설정 백업
ORIGIN_MIN=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].MinSize' --output text)
ORIGIN_MAX=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].MaxSize' --output text)
ORIGIN_DESIRED=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].DesiredCapacity' --output text)
echo "기존 용량: $ORIGIN_MIN(Min), $ORIGIN_MAX(Max), $ORIGIN_DESI
RED(Desired)"

# 2. 교체 대상 인스턴스 파악
instances=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].Instances[?!starts_with(LifecycleState,
`Terminating`)].InstanceId' \
  --output text)
read -ra instance_array <<< "$instances"
```

```
replace_count=${#instance_array[@]}
echo "교체 대상 인스턴스 ($replace_count개): ${instance_array[*]}"
print_instance_states

# 3. 용량 설정 변경 (교체할 인스턴스 수 + 1)
new_capacity=$((replace_count + 1))
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name "$ASG_NAME" \
  --min-size "$new_capacity" \
  --max-size "$new_capacity" \
  --desired-capacity "$new_capacity"
echo "용량 설정 변경: $replace_count → $new_capacity (Min=Max=Desir
ed)"

# 4. 새 인스턴스가 Healthy 상태가 될 때까지 대기
for i in $(seq 20); do
  echo "새 인스턴스 Healthy 대기: $i/20"
  healthy_count=$(aws autoscaling describe-auto-scaling-groups \
    --auto-scaling-group-names "$ASG_NAME" \
    --query "length(AutoScalingGroups[0].Instances[?LifecycleState=='In
Service' && HealthStatus=='Healthy'])" \
    --output text)
  echo "총 Healthy 인스턴스 수: $healthy_count/$new_capacity"
  # 상태 확인 완료
  if [[ "$healthy_count" -ge "$new_capacity" ]]; then
    echo "✅ Healthy 확인 완료"
    print_instance_states
    break
  fi
  # 상태 확인 실패
  if [ "$i" -eq 20 ]; then
    echo "❌ Healthy 확인 실패"
    print_instance_states
    restore_capacity
    exit 1
  fi
  sleep 10
done
```

```
# 5. 대상 인스턴스들을 하나씩 교체
echo "인스턴스 롤링 교체 시작..."

for instance_id in "${instance_array[@]}"; do

  # 5.1 종료 요청
  echo "$instance_id 종료 요청"
  aws autoscaling terminate-instance-in-auto-scaling-group \
    --instance-id "$instance_id" \
    --no-should-decrement-desired-capacity

  # 5.2 terminating 상태가 될 때까지 대기
  for i in $(seq 20); do
    echo "$instance_id Terminating 대기: $i/20"
    lifecycle=$(aws autoscaling describe-auto-scaling-groups \
      --auto-scaling-group-names "$ASG_NAME" \
      --query "AutoScalingGroups[0].Instances[?InstanceId=='$instance_i
d'].LifecycleState" \
      --output text)
    echo "$instance_id 상태: $lifecycle"
    # 상태 확인 완료
    if [[ "$lifecycle" == Terminating* || -z "$lifecycle" ]]; then
      echo "✅ Terminating 확인 완료"
      print_instance_states
      break
    fi
    # 상태 확인 실패
    if [ "$i" -eq 20 ]; then
      echo "❌ Terminating 확인 실패"
      print_instance_states
      restore_capacity
      exit 1
    fi
    sleep 10
  done

  # 5.3 새 인스턴스가 Healthy 상태가 될 때까지 대기
```

```
  for i in $(seq 20); do
    echo "새 인스턴스 Healthy 대기: $i/20"
    healthy_count=$(aws autoscaling describe-auto-scaling-groups \
      --auto-scaling-group-names "$ASG_NAME" \
      --query "length(AutoScalingGroups[0].Instances[?LifecycleState=='I
nService' && HealthStatus=='Healthy'])" \
      --output text)
    echo "총 Healthy 인스턴스 수: $healthy_count/$new_capacity"
    # 상태 확인 완료
    if [[ "$healthy_count" -ge "$new_capacity" ]]; then
      echo "✅ Healthy 확인 완료"
      print_instance_states
      break
    fi
    # 상태 확인 실패
    if [ "$i" -eq 20 ]; then
      echo "❌ Healthy 확인 실패"
      print_instance_states
      restore_capacity
      exit 1
    fi
    sleep 10
  done
done

# 6. 기존 용량 설정 복구
echo "롤링 업데이트 완료"
restore_capacity
```

기존 코드의 문제점

1. 인스턴스를 하나씩 교체해서, 시간 지연 발생

코드 개선점

1. rotate-web-instances.sh의 #5 부분을 “배치(batch)”로 묶어서 여러 대를 동시에 종료 → 새 인스턴스 기동 → 헬시 확인을 반복하는 것으로 변경

개선 코드

```
#!/bin/bash
set -Eeuo pipefail
echo "Rolling update ASG"
ASG_NAME="asg-k"
ORIGIN_MIN=0
ORIGIN_MAX=0
ORIGIN_DESIRED=0

# [추가] 배치·버퍼 파라미터(환경변수로 덮어쓰기 가능). 요구하신 기본값 유지
BATCH_SIZE=${BATCH_SIZE:-2}          # 한 번에 종료/교체할 인스턴스 수
BUFFER=${BUFFER:-$BATCH_SIZE}        # 여유 용량(원래 Desired + BUF
FER 만큼 확보)
TERMINATE_TIMEOUT=${TERMINATE_TIMEOUT:-300}  # [초] 배치 termi
nating 대기 타임아웃
HEALTHY_TIMEOUT=${HEALTHY_TIMEOUT:-600}      # [초] 배치 healthy
대기 타임아웃
SLEEP_SEC=${SLEEP_SEC:-6}                    # 폴링 주기

# 인스턴스 상태 출력 함수 (변경 사항 X)
print_instance_states() {
  echo "현재 인스턴스 상태:"
  aws autoscaling describe-auto-scaling-groups \
    --auto-scaling-group-names "$ASG_NAME" \
    --query 'AutoScalingGroups[0].Instances[*].[InstanceId, LifecycleStat
e, HealthStatus]' \
    --output text
}

# 기존 용량 복구 함수 (변경 사항 X)
restore_capacity() {
  aws autoscaling update-auto-scaling-group \
    --auto-scaling-group-name "$ASG_NAME" \
    --min-size "$ORIGIN_MIN" \
    --max-size "$ORIGIN_MAX" \
    --desired-capacity "$ORIGIN_DESIRED"
```

```
  echo "용량 복원 완료: $ORIGIN_MIN(Min), $ORIGIN_MAX(Max), $ORIGIN
_DESIRED(Desired)"
}

# 1. 기존 용량 설정 백업 (변경 사항 X)
ORIGIN_MIN=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].MinSize' --output text)
ORIGIN_MAX=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].MaxSize' --output text)
ORIGIN_DESIRED=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].DesiredCapacity' --output text)
echo "기존 용량: $ORIGIN_MIN(Min), $ORIGIN_MAX(Max), $ORIGIN_DESI
RED(Desired)"

# 2. 교체 대상 인스턴스 파악 (변경 사항 X)
instances=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].Instances[?!starts_with(LifecycleState,
`Terminating`)].InstanceId' \
  --output text)
read -ra instance_array <<< "$instances"
replace_count=${#instance_array[@]}
echo "교체 대상 인스턴스 ($replace_count개): ${instance_array[*]}"
print_instance_states

# 3. 용량 설정 변경 ( ORIGIN_DESIRED + BUFFER )
new_capacity=$((ORIGIN_DESIRED + BUFFER ))
aws autoscaling update-auto-scaling-group \
  --auto-scaling-group-name "$ASG_NAME" \
  --min-size "$new_capacity" \
  --max-size "$new_capacity" \
  --desired-capacity "$new_capacity"
echo "용량 설정 변경: Desired ${ORIGIN_DESIRED} → ${new_capacity}
(Min=Max=Desired)"
```

```
# 4. 새 인스턴스가 Healthy 상태가 될 때까지 대기
for i in $(seq $(( HEALTHY_TIMEOUT / SLEEP_SEC ))); do # 20대신 조절 가능한 파라미터 값으로 변경
  echo "새 인스턴스 Healthy 대기: $i"
  healthy_count=$(aws autoscaling describe-auto-scaling-groups \
    --auto-scaling-group-names "$ASG_NAME" \
    --query "length(AutoScalingGroups[0].Instances[?LifecycleState=='InService' && HealthStatus=='Healthy'])" \
    --output text)
  echo "총 Healthy 인스턴스 수: $healthy_count/$new_capacity"
  # 상태 확인 완료
  if [[ "$healthy_count" -ge "$new_capacity" ]]; then
    echo "✅ Healthy 확인 완료"
    print_instance_states
    break
  fi
  # 상태 확인 실패
  if (( i == HEALTHY_TIMEOUT / SLEEP_SEC )); then
    echo "❌ Healthy 확인 실패"
    print_instance_states
    restore_capacity
    exit 1
  fi
  sleep "$SLEEP_SEC"
done

# [추가] 기준 Healthy 수(초기 확장 후)를 기록 — 타깃 Healthy 계산에 사용
baseline_healthy=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query "length(AutoScalingGroups[0].Instances[?LifecycleState=='InService' && HealthStatus=='Healthy'])" \
  --output text)
echo "baseline_healthy: $baseline_healthy"

# 5. 대상 인스턴스들을 하나씩 교체
echo "인스턴스 롤링 교체 시작..."
replaced_total=0
```

```
total=${#instance_array[@]}

for ((start=0; start<total; start+=BATCH_SIZE)); do
  end=$(( start + BATCH_SIZE - 1 ))
  (( end >= total )) && end=$(( total - 1 ))
  batch_size=$(( end - start + 1 ))

  echo "-----------------------------------------------"
  echo "배치 종료 요청: index ${start} ~ ${end} (배치 크기: ${batch_size})"

  # 5.1 배치 동시 종료 요청(병렬) — [변경]
  pids=()
  for ((i=start; i<=end; i++)); do
    id="${instance_array[$i]}"
    echo "종료 요청 → $id"
    aws autoscaling terminate-instance-in-auto-scaling-group \
      --instance-id "$id" \
      --no-should-decrement-desired-capacity & pids+=($!)
  done
  # 모든 종료 API 호출 완료 대기
  for pid in "${pids[@]}"; do wait "$pid"; done

  # 5.2 배치 전체가 Terminating(or 목록에서 사라짐) 될 때까지 대기 — [변경]
  deadline=$(( SECONDS + TERMINATE_TIMEOUT ))
  while : ; do
    all_terminating=true
    for ((i=start; i<=end; i++)); do
      id="${instance_array[$i]}"
      lifecycle=$(aws autoscaling describe-auto-scaling-groups \
        --auto-scaling-group-names "$ASG_NAME" \
        --query "AutoScalingGroups[0].Instances[?InstanceId=='$id'].Lifec
ycleState" \
        --output text)
      if [[ -n "$lifecycle" && "$lifecycle" != Terminating* ]]; then
        all_terminating=false
        echo "$id 상태: $lifecycle (terminating 대기 중)"
      fi
    done
```

```
    $all_terminating && { echo "✅ 배치 terminating 확인 완료"; print_instan
ce_states; break; }
    (( SECONDS >= deadline )) && {
      echo "❌ 배치 terminating 확인 타임아웃"
      print_instance_states
      restore_capacity
      exit 1
    }
    sleep "$SLEEP_SEC"
  done

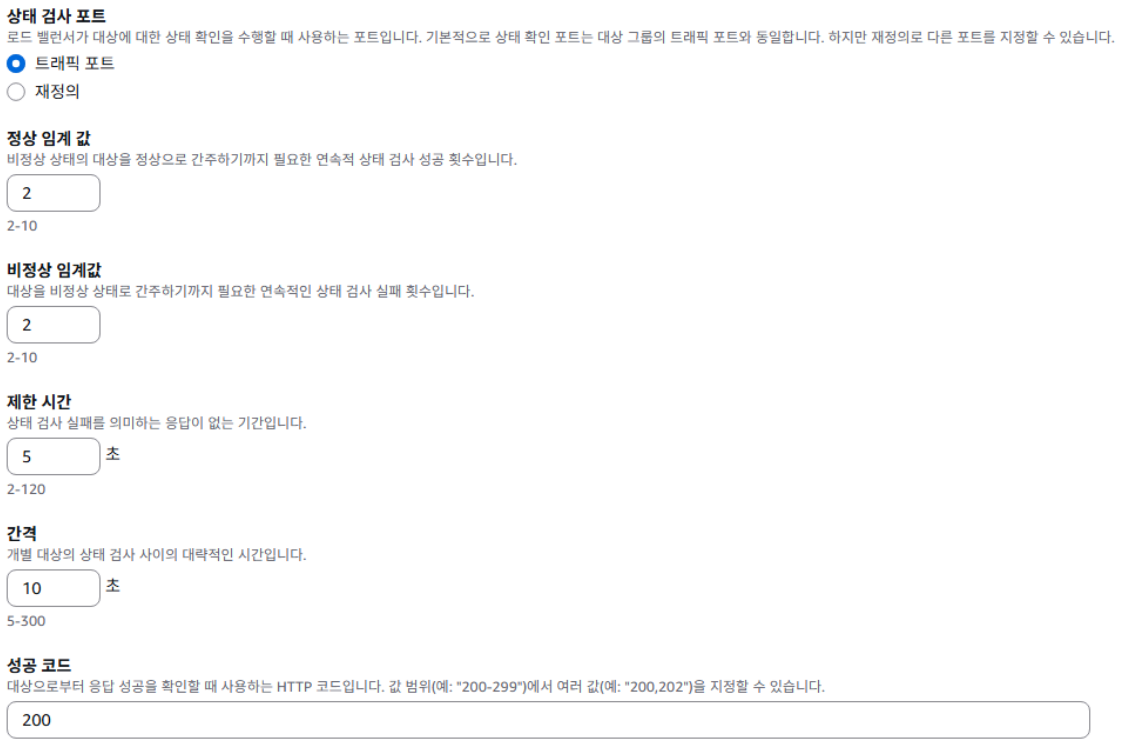
  # 5.3 새 인스턴스 Healthy 대기 — [변경]
  replaced_total=$(( replaced_total + batch_size ))
  target_healthy=$(( baseline_healthy + replaced_total ))
  if (( target_healthy > new_capacity )); then
    target_healthy=$new_capacity
  fi

  echo "새 인스턴스 Healthy 대기: 목표 ${target_healthy}/${new_capacit
y}"
  deadline=$(( SECONDS + HEALTHY_TIMEOUT ))
  while : ; do
    healthy_count=$(aws autoscaling describe-auto-scaling-groups \
      --auto-scaling-group-names "$ASG_NAME" \
      --query "length(AutoScalingGroups[0].Instances[?LifecycleState=='I
nService' && HealthStatus=='Healthy'])" \
      --output text)
    echo "현재 Healthy: ${healthy_count}/${new_capacity}"
    if (( healthy_count >= target_healthy )); then
      echo "✅ 배치 Healthy 확인 완료"
      print_instance_states
      break
    fi
    (( SECONDS >= deadline )) && {
      echo "❌ 배치 Healthy 확인 타임아웃"
      print_instance_states
      restore_capacity
      exit 1
```

```
    }
    sleep "$SLEEP_SEC"
  done
done

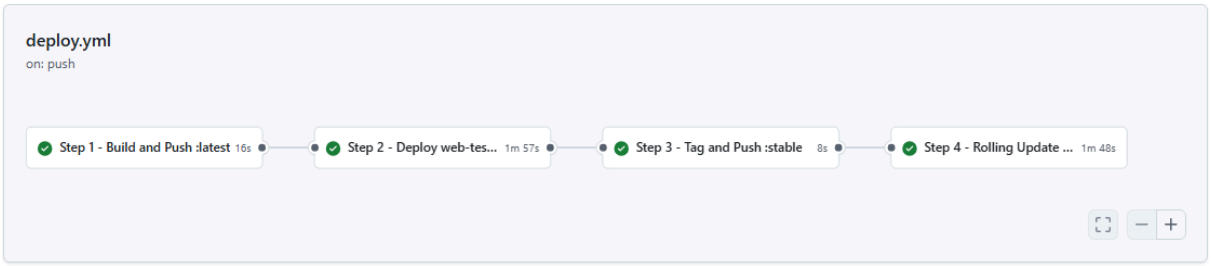
# 6. 기존 용량 설정 복구
echo "롤링 업데이트 완료"
restore_capacity
```

3. 추가 개선

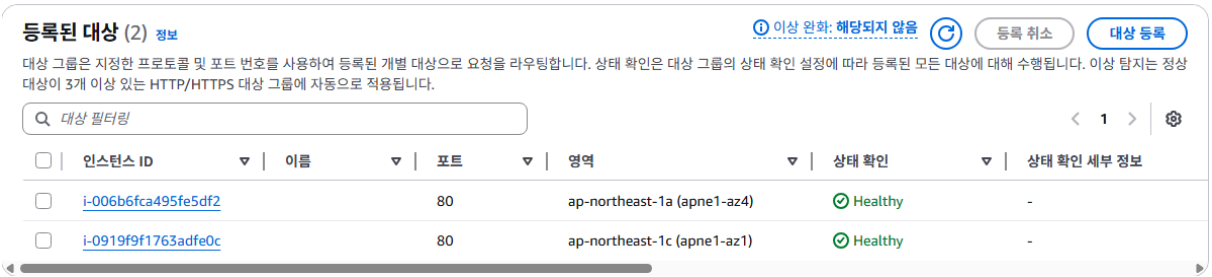


- 타겟 그룹 임계값 조정

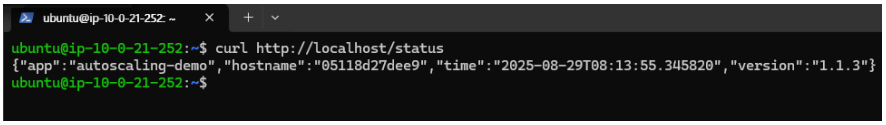
4. 개선 결과



- step 1: 16초
  step 2: **3분 6초 → 1분 57초**
  step 3: 11초
  step 4: **2분 12초 → 1분 58초**



- 기존 2~4분에서 1분 50초 ~ 2분 30초로 단축



- private 배포 완료

GitHub Actions scripts 개선

업무 영역	Devops 자동화
진행현황	대기
진행률	0
설명	1. rotate-instances.sh 개선
주차	4주차

0. 개요

- ASG로 생성된 인스턴스가 ALB 헬스체크를 하기 전에 Healthy 상태가 될 수 있다는 사실을 확인
- `rotate-instances-web.sh` 코드 일부 개선

1. 개선 내용

- 인스턴스 health check를 ALB 타겟 그룹에서 Healthy가 되었을 때로 변경
- Health Check 실행 시점을 맨 마지막에서 앞당겨, 이전 인스턴스를 Terminating하기 전에 완료되도록 순서를 재구성

2. 개선 코드

```
# [추가] ALB 타깃 그룹 Healthy 수 조회 헬퍼
healthy_targets() {
  aws elbv2 describe-target-health \
    --target-group-arn "$TG_ARN" \
    --region ap-northeast-1 \
    --query "length(TargetHealthDescriptions[?TargetHealth.State=='healthy'])" \
    --output text
}
```

```
# [추가] ASG에 연결된 첫 번째 Target Group ARN 확보
TG_ARN=$(aws autoscaling describe-auto-scaling-groups \
  --auto-scaling-group-names "$ASG_NAME" \
  --query 'AutoScalingGroups[0].TargetGroupARNs[0]' \
  --output text)
echo "사용할 Target Group: $TG_ARN"

# 4. [변경] 확장된 용량이 ALB(Target Group) Healthy 될 때까지 대기
for i in $(seq $(( HEALTHY_TIMEOUT / SLEEP_SEC ))); do
  echo "새 인스턴스 Healthy 대기: $i"
    tg_healthy=$(healthy_targets)  # ASG가 아닌 ALB 타겟 그룹의 Healthy 조회
  echo "TargetGroup Healthy: ${tg_healthy}/${new_capacity}"
  if (( tg_healthy >= new_capacity )); then
    echo "✅ 초기 확장 Target Group Healthy 확인 완료"
    print_instance_states
    break
  fi
  if (( i == HEALTHY_TIMEOUT / SLEEP_SEC )); then
    echo "❌ 초기 Target Group Healthy 확인 실패"
    print_instance_states
    restore_capacity
    exit 1
  fi
  sleep "$SLEEP_SEC"
done

# [변경] 기준 Healthy 수를 'ALB 타깃 그룹' 기준으로 기록
baseline_healthy=$(healthy_targets)
echo "baseline_healthy(TG): $baseline_healthy"

# 5.1, 5.2 변경 사항 X
# 5.3 삭제
# 6 변경 사항 X
```

```
# dw-bastion-role에 추가할 정책
{
  "Version": "2012-10-17",
```

```
  "Statement": [
    {
      "Sid": "AllowReadALBTargetHealth",
      "Effect": "Allow",
      "Action": [
        "elasticloadbalancing:DescribeTargetHealth",
        "elasticloadbalancing:DescribeTargetGroups",
        "elasticloadbalancing:DescribeLoadBalancers"
      ],
      "Resource": "*"
    }
  ]
}
```

3. 개선 결과

- 깃허브 액션과 AWS 콘솔의 인스턴스 상태가 다른 상황 제거
- Step 4의 수행 시간이 30초 ~ 1분 증가

